File Operations

1 Basic Idea

Definition

File operations are system calls or OS services that create, access, modify, inspect, and remove files.

- Files exist to store information.
- File operations allow information to be retrieved later.
- Different operating systems provide different sets of file operations.
- Most systems support a common core set of operations.
- These operations manage file data, file position, attributes, and names.

Core Point

File operations are the interface between programs and persistent file storage.

2 Common File Operations

Operation	Purpose
Create	Make a new empty file and set initial attributes.
Delete	Remove a file and free its disk space.
Open	Prepare a file for use by loading needed metadata into memory.
Close	Finish using a file and release internal OS table space.
Read	Copy data from a file into a caller-provided buffer.
Write	Copy data from a buffer into a file.
Append	Add data only at the end of a file.
Seek	Move the current file position.
Get attributes	Read file metadata such as size or modification time.
Set attributes	Change user-settable metadata such as protection flags.
Rename	Change the name of an existing file.

3 Create and Delete

Create

Create makes a new file with no data.

- The purpose is to announce that a file is coming.

- The OS creates the file-system entry.
- Initial attributes may be set at creation time.
- A newly created file may have size 0.

Delete

Delete removes a file when it is no longer needed.

- Deleting a file frees disk space.
- The file's directory entry or metadata reference is removed.
- Most operating systems provide a system call for deletion.

4 Open and Close

Open

Open prepares a file for later operations by bringing important file information into main memory.

- A process must usually open a file before using it.
- The OS fetches file attributes.
- The OS may fetch disk-address information.
- Keeping this information in memory makes later calls faster.
- The OS may create an entry in an open-file table.

Close

Close ends access to an open file and frees OS resources associated with it.

- Closing frees internal table space.
- Many systems limit how many files one process can keep open.
- Closing may force buffered data to be written to disk.
- Because disks write in blocks, the final partly filled block may be written at close time.

Resource Limit

Forgetting to close files can waste open-file table entries and may hit the process's maximum open-file limit.

5 Read and Write

Read

Read copies data from a file into a buffer supplied by the caller.

- Data usually come from the current file position.
- The caller specifies how many bytes are needed.
- The caller provides a buffer where the data should be placed.
- After the read, the current file position normally advances.

Write

Write copies data from a buffer into a file, usually at the current file position.

- If the current position is at the end of the file, the file grows.
- If the current position is in the middle of the file, existing data are overwritten.
- Overwritten data are lost unless they exist somewhere else.
- After the write, the current file position normally advances.

Overwrite Risk

Writing in the middle of a file replaces existing bytes. This can destroy old data permanently.

6 Append

Append

Append is a restricted form of write that adds data only to the end of a file.

- Append cannot overwrite existing data in the middle of the file.
- It is useful for logs and accumulating output.
- Minimal system-call interfaces may not include a separate append call.
- Some systems provide append anyway because it is convenient and safer for some uses.

7 Seek

Seek

Seek changes the current file position for a random-access file.

- Random-access files need a way to choose where reading or writing begins.
- `seek` moves the file pointer to a specific place in the file.

- After seeking, reads start from the new position.
- Writes also start from the new position.
- UNIX and Windows use this style of file positioning.

Seek Then Read

```
open(file);
seek(file, position);
read(file, buffer, nbytes);
close(file);
```

8 Get and Set Attributes

Attributes

File attributes are metadata such as protection bits, size, owner, and timestamps.

- Programs often need to read file attributes.
- Example: build tools compare modification times.
- The UNIX `make` program checks source-file and object-file timestamps.
- If a source file is newer than its object file, recompilation is needed.
- Some attributes can be changed by users.
- Protection mode and many flags are common user-settable attributes.

Operation	Example use
Get attributes	Read modification time, file size, owner, or protection bits.
Set attributes	Change protection mode, read-only flag, or other user-settable flags.

9 Rename

Rename

Rename changes the name of an existing file.

- Users frequently need to rename files.
- A rename operation changes the file name without rewriting all file data.
- Rename is not always strictly necessary.
- A file can usually be copied to a new file with the new name.
- Then the old file can be deleted.
- Direct rename is more efficient and more convenient.

10 Operation Groups

Group	Operations
Lifecycle	Create, delete, open, close.
Data transfer	Read, write, append.
Positioning	Seek.
Metadata	Get attributes, set attributes.
Naming	Rename.

11 Typical File Use Flow

1. Create or locate the file.
2. Open the file.
3. Read, write, append, or seek as needed.
4. Get or set attributes if needed.
5. Close the file.
6. Delete or rename the file when appropriate.

Final Point

File operations manage the file's lifetime, contents, current position, metadata, and name.